Lesson 12: Hearing It Happen

TIC-80

VIDEO GAME CREATION

This week: your game finally sounds like a game. Four moments - firing, hitting, clearing a wave, and losing - each get their own distinct cue.

Open your own [YourName]Game project from Lesson 11 - don't start a new one.

If you already built an optional sound idea in an earlier lesson (Lessons 2, 3, 4, or 6), keep it - today reuses it directly instead of replacing it. If you didn't, today's steps are written as if starting fresh.

Setup

1. Open your [YourName]Game project from Lesson 11.
2. Design one new short SFX in the SFX Editor (F4): slot 6 (wave clear) - distinct from your existing slot 1 (shoot), slot 2 (hit), slot 3 (win, if built), slot 4 (life lost), and slot 5 (game over).

Level 1 - Minimal (MVP)

Four sounds fire at the correct moment, at least once each: a shot, a hit/explosion, a wave-clear (on any non-final wave), and a game over. No silence, no obviously wrong triggers.

3. Confirm sfx(1) already fires in your shoot branch (Lesson 3) - add it now if it's missing.
4. In the bullets loop's hit branch, confirm sfx(2) already fires when a hit is detected (Lesson 4/5) - add it now if it's missing.
5. In that same branch's nested if (from Lesson 11), add sfx(3) in the true-win case (only if you built it) and sfx(6) in the else (wave-advance) case.
6. In the enemy movement loop's escape branch, confirm sfx(5) already fires inside the lives<=0 check (from Lesson 8) - add it now if it's missing. This sound is mandatory starting today even if you skipped it before.
7. Press Esc, type run, press Enter, then play through a full round - shoot, hit, an early wave clear, and a loss should each sound distinct.

Highlighted lines are new this week; everything else is code you already have.

```
-- bullets loop's hit branch (revised):
score = score + 10
enemies_remaining = enemies_remaining - 1
sfx(2)
if enemies_remaining <= 0 then
 if wave >= total_waves then
  game_active = false
```

```
  has_won = true
  sfx(3) -- your Lesson 6 Level 3 sound
 else
  wave = wave + 1
  enemies_remaining = 5
  sfx(6)
  spawn_wave()
 end
end

-- enemy movement/escape loop (revised):
for enemy_index=1,#enemies do
 local enemy=enemies[enemy_index]
 if enemy.alive then
  enemy.ey=enemy.ey+0.3
  if enemy.ey>136 then
    enemy.alive=false
    lives=lives-1
    if lives<=0 then
      game_active=false
      has_lost=true
      sfx(5) -- your Lesson 8 Level 3 sound, mandatory as of today
    else
      sfx(4) -- your Lesson 8 Level 3 sound
    end
  else
    spr(3,enemy.ex,enemy.ey,0)
  end
 end
end
```

Level 2 - Complete (Alpha/Beta)

All four sounds fire reliably every time, with no missing or duplicate playback, and each is clearly distinguishable from the others by ear.

- Play a full round: shoot, hit, an early wave clear, and a loss each produce a distinct sound.
- If you built the optional win sound, confirm it still plays on the true win, and sfx(6) doesn't also fire on that same frame.
- None of today's sfx() calls need an sfx(-1) stop call or a replay-guard flag - each lives inside a branch that only executes once per actual hit or escape event, not a per-frame poll.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Audio: if you haven't already, design slot 3 and add sfx(3) now - it slots into the exact same nested if as today's wave sound, just the opposite branch.
- Audio: design a second hit sound in an empty slot and alternate between the two using math.random(0,1), so repeated hits don't sound completely identical.
- Structure: label your six SFX Editor slots with a comment at the top of your code (e.g. -- slots: 1=shoot 2=hit 3=win 4=lifelost 5=gameover 6=wave) so the mapping stays clear as the project grows.

If Something's Wrong

- Wrong sound plays: confirm each sfx() call uses the slot number matching what you designed in the SFX Editor - it's easy to mix up adjacent slots.
- No sound on wave-clear: confirm sfx(6) is actually inside the else branch, not after the if/else.
- Game-over sound plays repeatedly: confirm the escape branch's lives<=0 check is still a one-time check (from Lesson 8), not re-triggering every frame.

Want More?

Student Resource List > TIC-80 section, or github.com/nesbox/TIC-80/wiki